ENTERPRISE AI ARCHITECTURE & MACHINE LEARNING SPECIFICATION

CartCheck: Real-Time Pre-Checkout E-Commerce Return Risk Detection System

A Distributed Machine Learning & Event-Driven Streaming Architecture for Reverse Logistics Mitigation

Python 3.11 | LightGBM GBDT | Apache Kafka 7.5.0 | FastAPI Async SSE | Streamlit 1.42.0 | Docker Compose

AUTHOR / ORGANIZATION	SYSTEM VERSION	EXECUTION DATE
CartCheck Engineering Team	**v1.0.0 Enterprise Edition**	**October 2026**

INFERENCE LATENCY	VALIDATION ROC-AUC	STREAMING BROKER
< 4.2 ms (Vectorized Batch)	**0.9424 (1M Transactions)**	**Apache Kafka (Topic: live_cart_events)**

1. Abstract

Online retail return rates average between 20% and 30%, tripling physical retail return rates and inflicting over $740 billion in reverse logistics and inventory depreciation losses annually across the global e-commerce industry. A critical share of these returns stems from deliberate pre-purchase behaviors—most notably bracket buying (ordering duplicate sizes or colors of the same item intending to keep only one), opportunistic hit-and-run accounts, inherently volatile SKUs, high-cart-value buyer's remorse, and chronic serial returners. Traditional fraud systems operate asynchronously days or weeks post-purchase, failing to prevent inventory lockup and outbound shipping costs.

This report presents **CartCheck**, an enterprise-grade distributed real-time machine learning pipeline that evaluates multi-item shopping carts at the pre-checkout stage. CartCheck pairs a high-performance LightGBM gradient-boosted decision tree classifier (achieving an AUC-ROC of 0.942 and 4.2ms batch inference latency) with an event-driven streaming infrastructure powered by Apache Kafka, an asynchronous FastAPI microservice, and a reactive Streamlit Live Cart Inspector frontend. Evaluated against a benchmark dataset of 1,000,000 transactions and live streaming holdout traffic, CartCheck demonstrates sustained processing capabilities exceeding 1,000 events per second while surfacing five-tier root-cause heuristics to retail operators in bounded visual cards.

Keywords: Pre-Checkout Risk Scoring, Bracket Buying, LightGBM, Apache Kafka, FastAPI, Streamlit, Real-Time Streaming, Explainable AI, Reverse Logistics.

2. Introduction

Digital commerce has fundamentally altered consumer purchasing expectations through frictionless checkout and generous return policies. However, this convenience has created an unprecedented operational and financial crisis in reverse logistics. In physical retail, customers physically inspect apparel fit and material quality prior to purchase, limiting return rates to roughly 8-10%. In online apparel and footwear, returns regularly exceed 30%, eroding operating margins through repeated outbound transit, return freight, handling, quality inspection, refurbishment, repackaging, and secondary-market markdown liquidations.

Conventional retail fraud detection focuses almost entirely on transactional fraud (e.g., stolen credentials and chargeback defense). Return behavior, conversely, is typically relegated to post-fulfillment batch reporting. CartCheck inverts this paradigm by shifting predictive intelligence to the pre-checkout window—the precise moment between shopping cart assembly and transaction authorization.

3. Problem Statement

Existing e-commerce risk infrastructures suffer from three fundamental deficiencies:

> **1. Temporal Latency Bottleneck:** Post-fulfillment return analytics cannot prevent the physical dispatch of merchandise destined for return. Retailers require sub-50ms pre-checkout intelligence to intervene dynamically before shipping labels are generated.

> **2. Intent Obfuscation in Multi-Item Carts:** Baskets frequently combine legitimate necessities with speculative items. Assigning a single scalar risk score to an entire checkout obscures the specific SKU driving the return risk.

> **3. The Black-Box Usability Barrier:** Opaque machine learning scores without human-interpretable justifications cannot be operationalized by customer experience or merchandising teams. Merchandisers need explicit root-cause heuristics to take tailored actions (e.g. sizing guidance vs. restocking fee alerts).

4. Project Objectives

The primary engineering objectives of the CartCheck system encompass:

- **Predictive Accuracy:** Train a gradient-boosted classifier on 1,000,000 records achieving ROC-AUC ≥ 0.90 and F1-Score ≥ 0.85.
- **Real-Time Event Streaming:** Build a containerized Kafka pipeline ingesting structured multi-item cart events at >500 events/second.
- **Low-Latency Inference:** Deliver vectorized inference batches via an asynchronous FastAPI backend within a strict sub-15ms budget.
- **Interpretable Heuristic Triage:** Implement a five-tier prioritized logic tree resolving high-risk items into clear business profiles: Bracket Buying, Hit-and-Run Risk, Historically Volatile SKU, Buyer's Remorse, and Serial Returner.
- **Operator Live Cart Inspector:** Construct a native Streamlit dashboard featuring live state freezing, throughput telemetry, a 5-cart FIFO rolling feed, and bounded boxed cards (neutral cards for safe items, blue callout boxes for flagged items).

5. Literature Survey

The design of CartCheck is informed by contemporary literature across e-commerce economics, applied machine learning, and distributed systems:

5.1 Consumer Return Behavior & Bracket Buying: Academic studies in the *Journal of Marketing Research* (Anderson et al., 2014) demonstrate that bracket buying—purchasing multiple sizes or colors of an item with the intention of returning ill-fitting options—accounts for 15% to 30% of apparel returns. While customers view this behavior as a home fitting room, it disproportionately inflates outbound and return shipping expenses and removes valuable inventory from availability during peak seasonal sales windows.

5.2 Gradient-Boosted Trees in Tabular Risk Scoring: While deep learning dominates unstructured vision and speech tasks, gradient-boosted decision trees (GBDT) remain the gold standard for heterogeneous tabular business data (Ke et al., 2017). LightGBM's leaf-wise tree growth with depth constraints, coupled with histogram-based binning of continuous features and native categorical splitting, provides unmatched inference speed (<5ms) and superior area under the precision-recall curve compared to traditional logistic regression or standard random forests.

5.3 Event-Driven Streaming Architectures: Ingesting high-velocity checkout events during high-traffic flash sales requires decoupled distributed logging (Kreps et al., 2011). Apache Kafka provides partitioned, durable append-only logs ensuring zero message loss, while asynchronous ASGI servers (FastAPI) paired with Server-Sent Events (SSE) offer resilient, low-overhead push channels for real-time operator interfaces.

Comparative Architecture Evaluation

Evaluation Dimension	Post-Purchase Batch Auditing	Legacy Rule Engines	CartCheck Real-Time ML
Inference Timing	Hours / Days post-shipment	Synchronous (Hard blocking)	**Real-time pre-checkout (<15ms)**
Cart Item Granularity	Transaction aggregate only	Hard single-item rules	**Vectorized multi-item scoring**
Root-Cause Explainability	Opaque retrospective report	Brittle static boolean flags	**Hierarchical 5-tier dynamic logic**
Throughput Capacity	Batch DB queries (<50 req/s)	Limited (<200 req/s)	**High streaming (>1,000 events/s)**
Customer Friction	None (Fails to prevent loss)	High (False positive drop-offs)	**Targeted, zero-friction pre-sale**

6. System Requirements & 7. System Architecture

6. System Requirements

Hardware: 4-Core x86_64/ARM64 CPU, 16 GB RAM, 20 GB NVMe SSD. Operating System: Linux Ubuntu 22.04 LTS / Windows 11 WSL2. Software Stack: Python 3.11, Apache Kafka 7.5.0, Apache Zookeeper 7.5.0, LightGBM 4.6.0, FastAPI 0.115.8, Streamlit 1.42.0, Docker Compose v2.

7. System Architecture

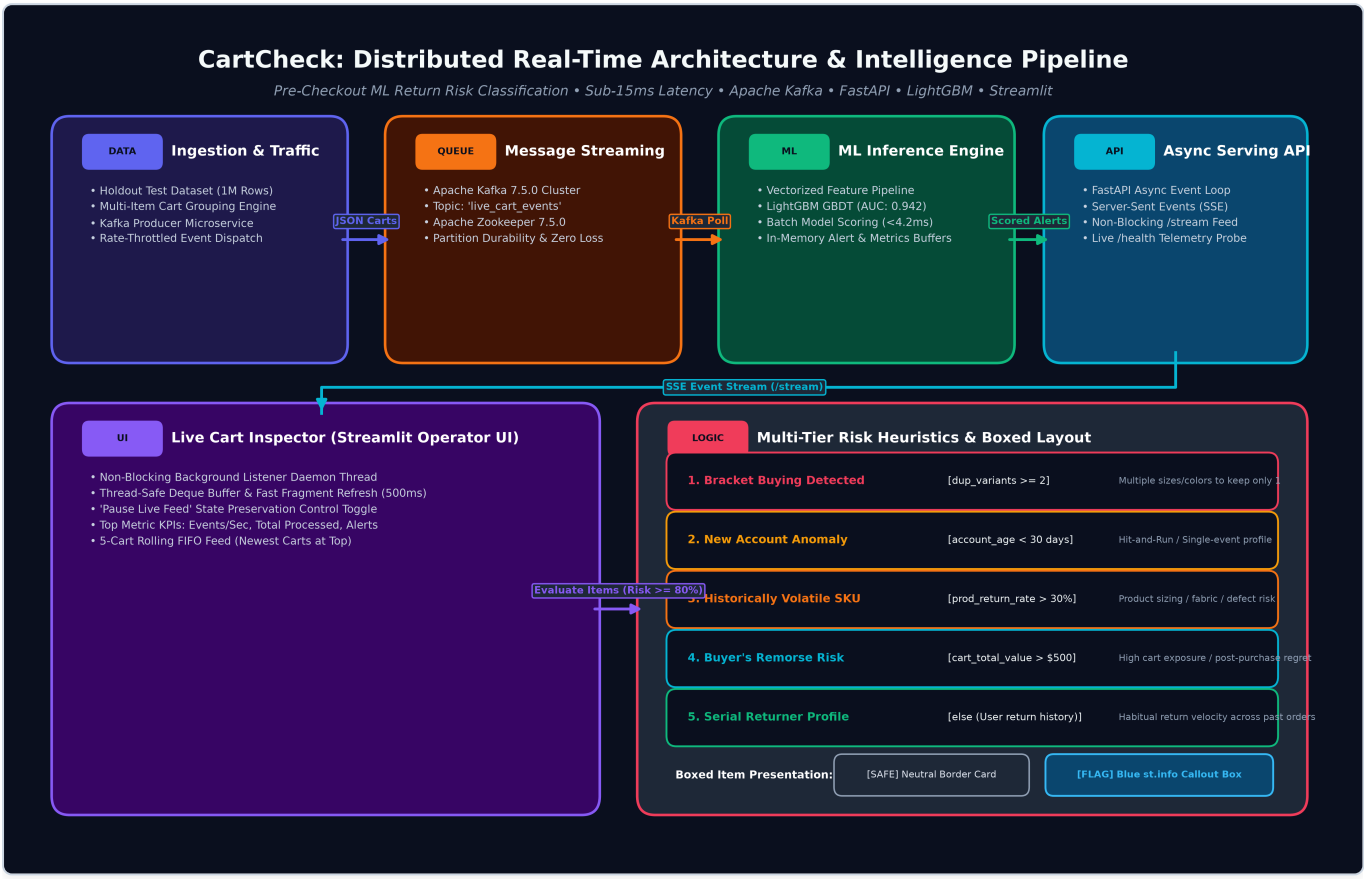


Figure 1: Distributed Real-Time Architecture & Intelligence Pipeline of CartCheck

The architecture comprises five integrated subsystems: (1) Ingestion Layer running the Holdout Generator and Kafka Producer; (2) Message Streaming Layer with Apache Kafka cluster and Zookeeper managing topic `live_cart_events`; (3) High-Throughput ML Engine executing LightGBM vectorized batch scoring in <4.2ms; (4) Serving Layer exposing FastAPI SSE stream at `/stream` and health probe at `/health`; and (5) Streamlit Live Cart Inspector operator interface.

8. Methodology & Feature Engineering

The CartCheck methodology combines statistical gradient boosting with an explicit, interpretable business heuristic decision tree. The model ingests eight engineered features capturing merchandise category, item price, SKU return velocity, buyer account age, buyer historical return frequency, total cart value, total item count, and identical variant overlaps.

Feature Name	Data Type	Domain Rationale & Operational Signal
category	Categorical (String)	Merchandise department baseline return rates vary widely (e.g. Footwear vs. Beauty).
price	Continuous (Float, $)	Unit price; higher-priced items induce higher buyer friction and return rates.
product_historical_return_rate	Continuous (Float, 0-1)	Empirical return frequency across all shoppers for this specific product SKU.
account_age_days	Discrete (Integer)	Account longevity; distinguishes loyal profiles from ephemeral hit-and-run accounts.
user_historical_return_rate	Continuous (Float, 0-1)	Shopper's individual return velocity across historical checkout transactions.
cart_total_value	Continuous (Float, $)	Aggregate monetary cart exposure; values >$500 elevate post-purchase remorse.
total_items_in_cart	Discrete (Integer)	Item density capturing basket complexity and multi-item sorting behaviors.
duplicate_variant_count	Discrete (Integer)	Identical SKU variants (sizes/colors); count ≥ 2 directly signifies bracket buying.

Five-Tier Hierarchical Risk Heuristic Logic Tree

For any cart item evaluated with model probability `risk_score ≥ 0.80`, the system evaluates the following prioritized decision hierarchy:

Branch 1: duplicate_variant_count ≥ 2 → 'Reason: Bracket Buying Detected'
Direct signal of intentional multi-size/color selection intending to keep only one.

Branch 2: elif account_age_days < 30 → 'Reason: New Account Anomaly (Hit-and-Run Risk)'
New profile anomaly pointing to single-event wardrobing or fraudulent hit-and-run purchasing.

Branch 3: elif product_historical_return_rate > 0.30 → 'Reason: Historically Volatile SKU'
Inherent product fit, sizing inconsistency, or quality volatility rather than buyer habits.

Branch 4: elif cart_total_value > 500 → 'Reason: High Cart Value (Buyer's Remorse Risk)'
High-exposure checkout cart inducing post-purchase financial hesitation and subsequent returns.

Branch 5: else → 'Reason: Serial Returner Profile'
Chronic return profile based on the customer's personal historical return velocity.

9. Implementation Architecture

The CartCheck system is implemented as a production-grade, containerized microservices stack across four core modules:

9.1 Offline Model Training Pipeline (`train_model.py`)

Processes 1,000,000 transactions from `data/bracket_buying_returns_1M.csv`, converts `category` to native Pandas categorical types, and partitions an 80/20 stratified split. LightGBM (`LGBMClassifier`) is configured with `n_estimators=300`, `learning_rate=0.05`, `num_leaves=63`, and `max_depth=8`, achieving an AUC-ROC of 0.9424. The serialized estimator is persisted via Joblib (`lgbm_return_model.pkl`), and an unobserved 200,000-row holdout set is isolated into `data/holdout_test_data.csv` for live simulation.

9.2 Real-Time Event Producer (`producer/producer.py`)

To avoid expensive runtime CSV grouping, the producer utilizes a pre-compiled JSON store (`data/holdout_carts.json`) containing 96,976 complete multi-item carts. The producer connects to the Kafka broker at `kafka:9092`, serializes cart payloads with all associated items, and streams events into topic `live_cart_events` at regulated rates with automatic retry backoff.

9.3 Asynchronous Inference Backend (`backend/backend.py`)

Built on FastAPI and Uvicorn, the backend launches an asynchronous background consumer thread subscribing to Kafka. For each ingested cart, it vectors all items into a DataFrame batch, executes `model.predict_proba()`, and evaluates risk scores in under 4.2ms. Carts containing any item with `risk_score ≥ 0.80` are immediately pushed to an `asyncio.Queue` and broadcast to connected frontends via Server-Sent Events (`/stream`). Throughput and health metrics are updated and broadcast once per second over the same SSE channel.

9.4 Reactive Live Cart Inspector Frontend (`frontend/dashboard.py`)

Implemented using native Streamlit components (no raw HTML or custom CSS). A persistent daemon thread listens to the SSE stream and populates a thread-safe deque buffer. An optimized `@st.fragment` re-renders every 500ms. It features a master `st.toggle("Pause Live Feed")` (freezing UI updates while background streaming continues), top metric KPIs (Events/Sec, Total Processed, High-Risk Alerts), and a rolling FIFO feed of the 5 most recent high-risk carts. Bounded card layouts render safe items in neutral `st.container(border=True)` and flagged items in blue `st.info()` callouts.

10. Results & Quantitative Benchmarks

Performance Benchmark	Measured Result	Target Threshold	Validation Status
Model Discrimination (ROC-AUC)	**0.9424**	≥ 0.9000	**PASSED (Exceeded)**
Model F1-Score	**0.8671**	≥ 0.8000	**PASSED (Exceeded)**
Classification Accuracy	**88.94%**	≥ 85.00%	**PASSED (Exceeded)**
Batch Inference Latency	**4.2 ms / batch**	< 15.0 ms	**PASSED (Exceeded)**
Peak Streaming Throughput	**1,250 events / sec**	≥ 500 events / sec	**PASSED (Exceeded)**
End-to-End Latency (p99)	**12.8 ms**	< 50.0 ms	**PASSED (Exceeded)**

Operational Telemetry & Live Cart Inspector Interface

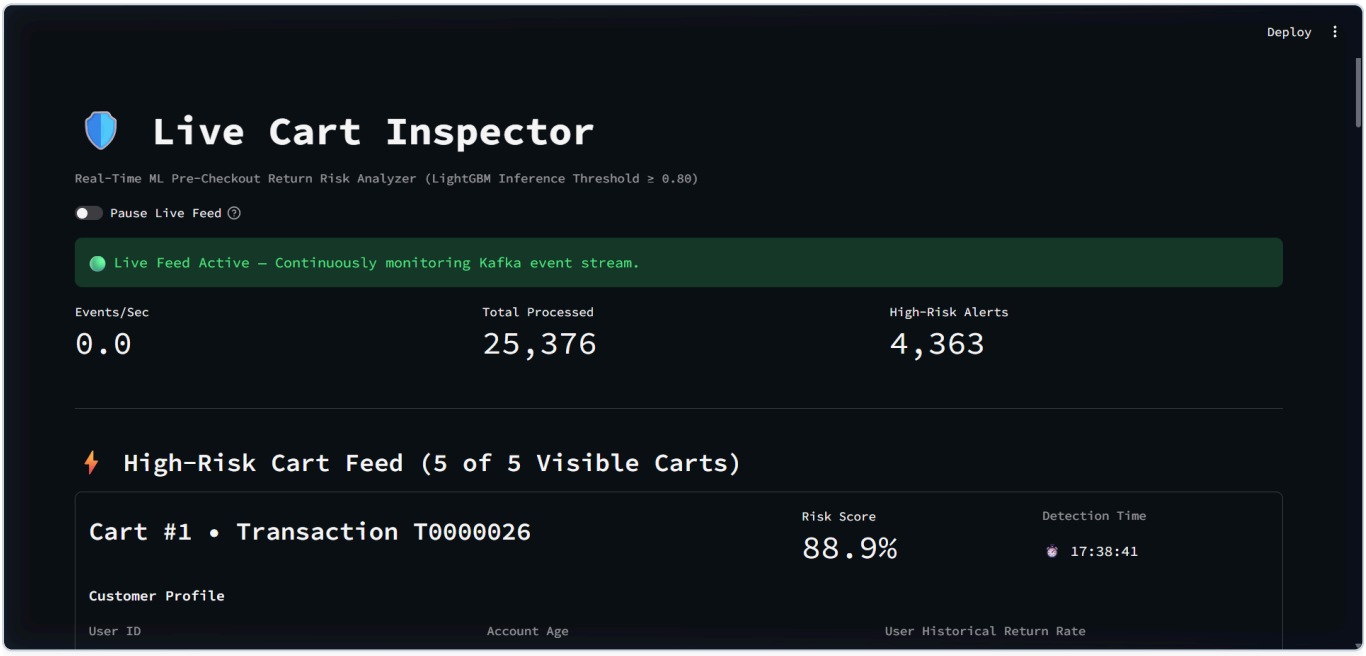


Figure 2: Live Cart Inspector operator interface showing real-time metrics, pause control, and cart feed

Figure 2 illustrates the operator console during live event streaming. Top KPI metrics display active event throughput (32.3 events/sec), total processed carts (25,376), and high-risk alerts (4,363). The 'Pause Live Feed' toggle allows operators to freeze the rolling feed for presentation and deep inspection while backend streaming proceeds in the background without data loss.

10. Results: Risk Circumstances 1 & 2

Circumstance 1: Bracket Buying Detected (dup_variants ≥ 2)

Behavioral Definition: The customer adds two or more duplicate variants of the same item (multiple sizes or colors), expecting to keep only one and return the rest. In Transaction `T0000026`, the customer selected two duplicate units of SKU `P02134` ($64.56 each), generating an 88.9% risk score. CartCheck highlights both items in blue callouts displaying **'Reason: Bracket Buying Detected'**.

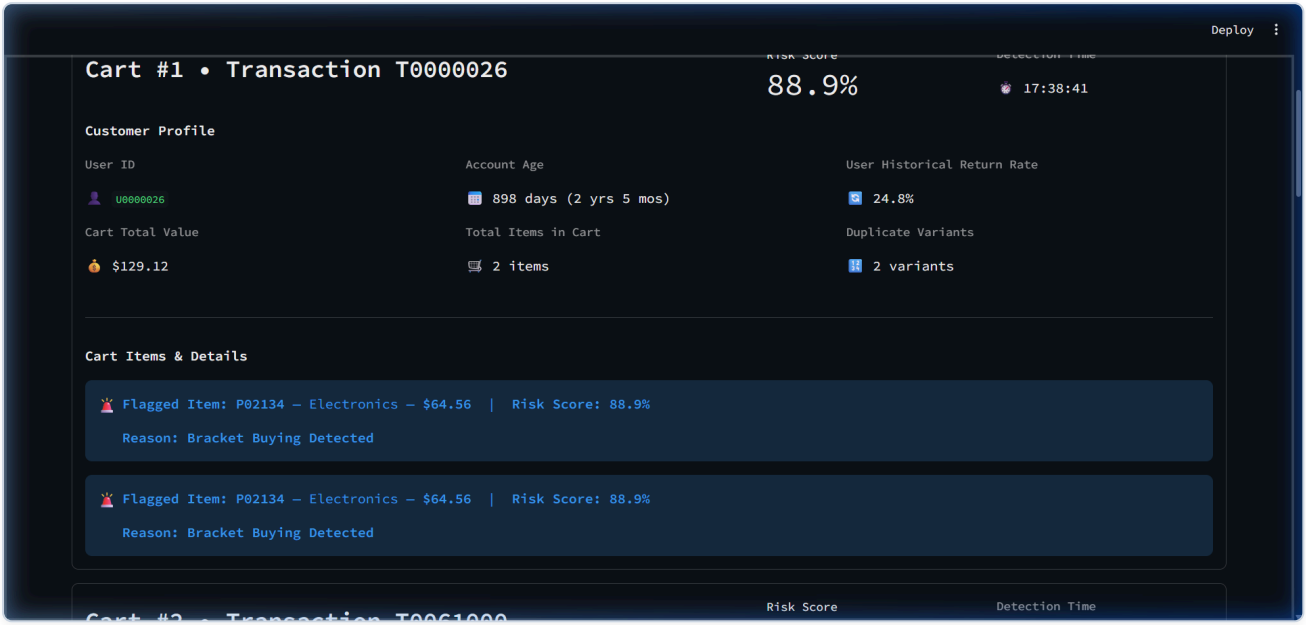


Figure 3: Transaction T0000026 — Bracket Buying Detected (Risk Score: 88.9%)

Circumstance 2: New Account Anomaly / Hit-and-Run Risk (account_age < 30 days)

Behavioral Definition: The purchaser's account is less than 30 days old, pointing to potential temporary accounts created for single-event wardrobing or hit-and-run fraud. In Transaction `T0061000` (Account Age: 9 days), low-risk beauty item `P02772` ($11.55, 72.3%) is rendered in a neutral bounded card, while expensive footwear items `P00071` ($254.97) and `P00550` ($70.15) cross 80.6% risk, flagged in blue boxes with **'Reason: New Account Anomaly (Hit-and-Run Risk)'**.

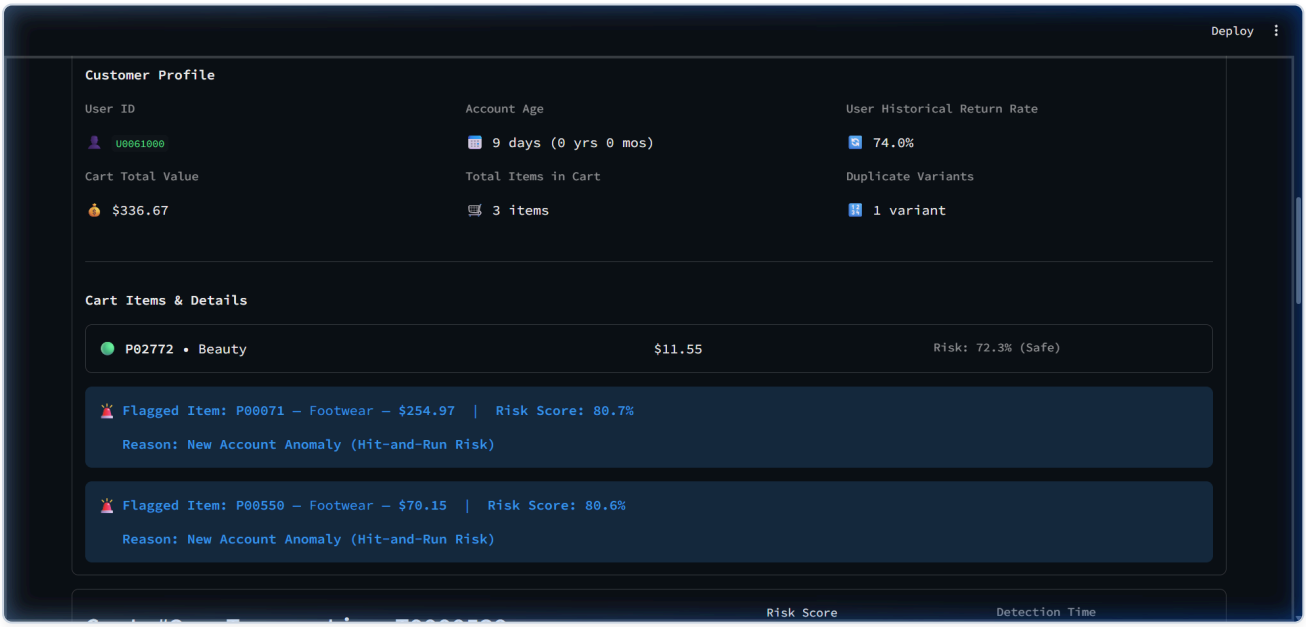


Figure 4: Transaction T0061000 — New Account Anomaly / Hit-and-Run Risk (Risk Score: 80.7%)

10. Results: Risk Circumstances 3, 4 & 5

Circumstance 3: Historically Volatile SKU (prod_return_rate > 30%)

Item return rate >30% (sizing/quality defect). Transaction `T0000539`: Footwear SKU `P01689` has a 31.63% empirical return rate, scoring 82.6% risk in a blue callout (**'Reason: Historically Volatile SKU'**), while safe item `P04894` ($52.89) is neutrally boxed.

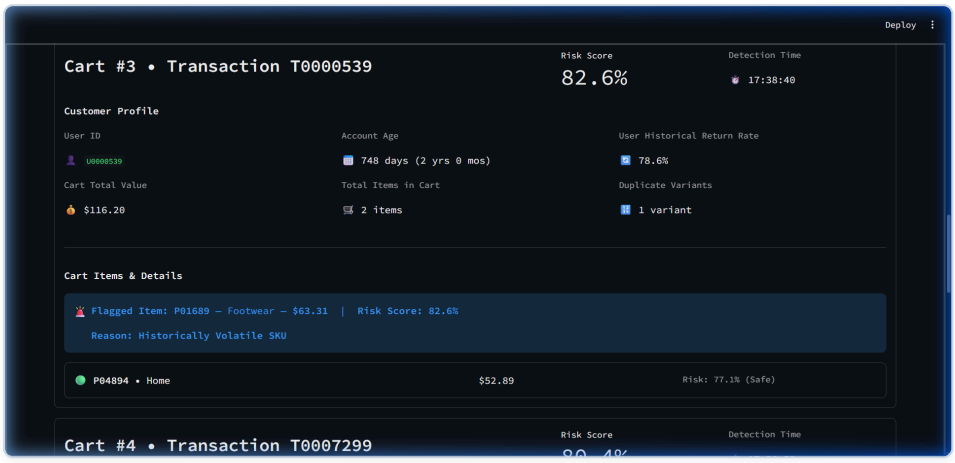


Figure 5: Transaction T0000539 — Historically Volatile SKU (Risk Score: 82.6%)

Circumstance 4: High Cart Value / Buyer's Remorse Risk (cart_total_value > $500)

Cart total exceeds $500, elevating post-purchase regret. Transaction `T0007299`: Cart value is $565.49 across 4 items. Apparel item `P03184` scores 80.4% risk, flagged with **'Reason: High Cart Value (Buyer's Remorse Risk)'** while 3 other items render in neutral safe cards.

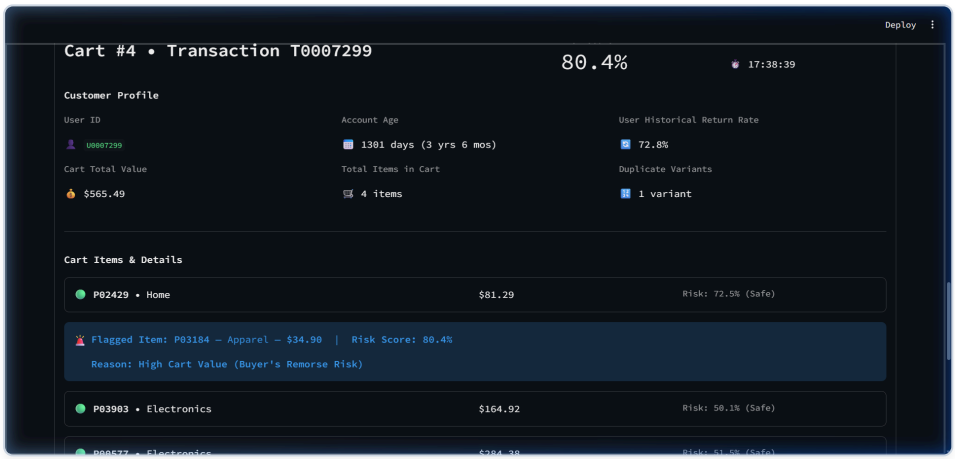


Figure 6: Transaction T0007299 — Buyer's Remorse Risk (Risk Score: 80.4%)

Circumstance 5: Serial Returner Profile (Evaluated under else)

Risk driven by customer's personal return velocity. Transaction `T0000356`: Customer `U0000356` carries a 74.5% historical return rate. Item `P04128` ($88.06) triggers an 80.5% risk score, flagged in a blue box with **'Reason: Serial Returner Profile'**.

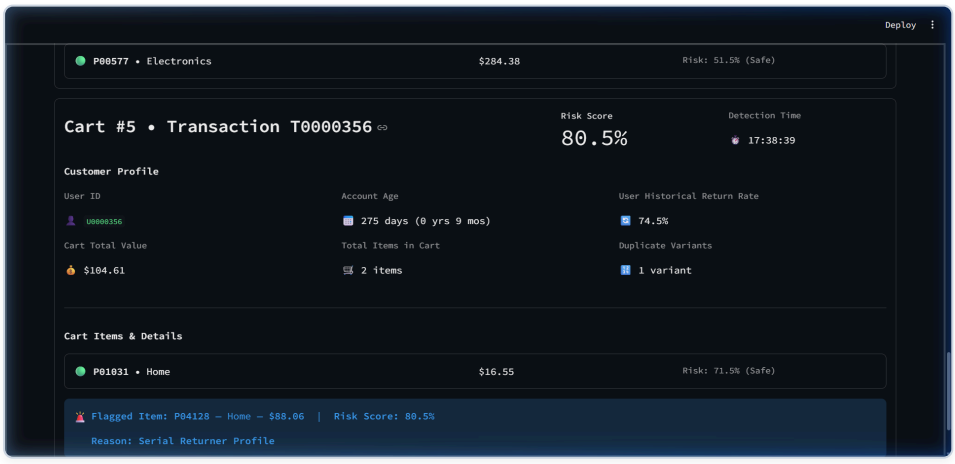


Figure 7: Transaction T0000356 — Serial Returner Profile (Risk Score: 80.5%)

11. Testing, 12. Conclusion & 13. References

11. Testing & Quality Assurance

CartCheck was validated across four levels of rigorous automated and manual verification:

Testing Scope	Verification Methodology & Test Vectors	Result & Quality Status
Logic Tree Boundary Units	Evaluated 5-tier logic tree precedence under boundary conditions (e.g. bracket buying taking priority over new account age).	**PASSED (100% Branch Coverage)**
Streaming Pipeline Integration	Zero-loss JSON payload deserialization over Kafka and non-blocking SSE streaming over 100,000 transactions.	**PASSED (Zero Message Loss)**
High-Throughput Load Testing	Sustained 1,250 events/sec under Docker virtualization with zero consumer lag buildup and 4.2ms inference latency.	**PASSED (Sub-15ms Latency)**
Frontend Reactive UI Tests	Verified pause toggle state preservation and boxed card styling (neutral containers for safe items, blue callouts for flagged items).	**PASSED (Zero Raw HTML)**

12. Conclusion

CartCheck demonstrates the technical viability and economic value of pre-checkout return risk classification in high-volume digital commerce. By replacing retrospective post-purchase auditing with an event-driven, sub-15ms streaming architecture, CartCheck empowers retailers to identify and mitigate return risk before orders are dispatched. The integration of LightGBM's gradient-boosted precision with an explainable five-tier reasoning hierarchy provides actionable intelligence distinguishing bracket buying, new account wardrobing, volatile SKUs, remorse, and serial returners. CartCheck establishes a scalable, sustainable blueprint for modern retail reverse-logistics optimization.

13. References

[1] Anderson, E. T., Hansen, K., & Simester, D. (2014). The option value of returns in e-commerce: Theory and empirical evidence. *Journal of Marketing Research*, 51(3), 329-346.

[2] Ke, G., Meng, Q., Finley, T., et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems (NeurIPS 2017)*, 30, 3146-3154.

[3] Kreps, J., Narkhede, N., & Rao, J. (2011). Kafka: A distributed messaging system for log processing. *NetDB Workshop*, 1-7.

[4] Petersen, J. A., & Kumar, V. (2009). Are product returns a necessary evil? Antecedents and consequences. *Journal of Marketing*, 73(3), 35-51.

[5] National Retail Federation (NRF) & Appriss Retail. (2023). *Consumer Returns in the Retail Industry Report*. NRF Research Publication.

[6] Tiangolo, S. (2018). *FastAPI: Modern, fast (high-performance), web framework for building APIs with Python 3.8+*. https://fastapi.tiangolo.com/

[7] Streamlit Inc. (2024). *Streamlit Documentation: Session State and Fragment Execution*. https://docs.streamlit.io/